

# Complete Data Flow

From Simulation to Doctor Dashboard — Including the LLM Stack

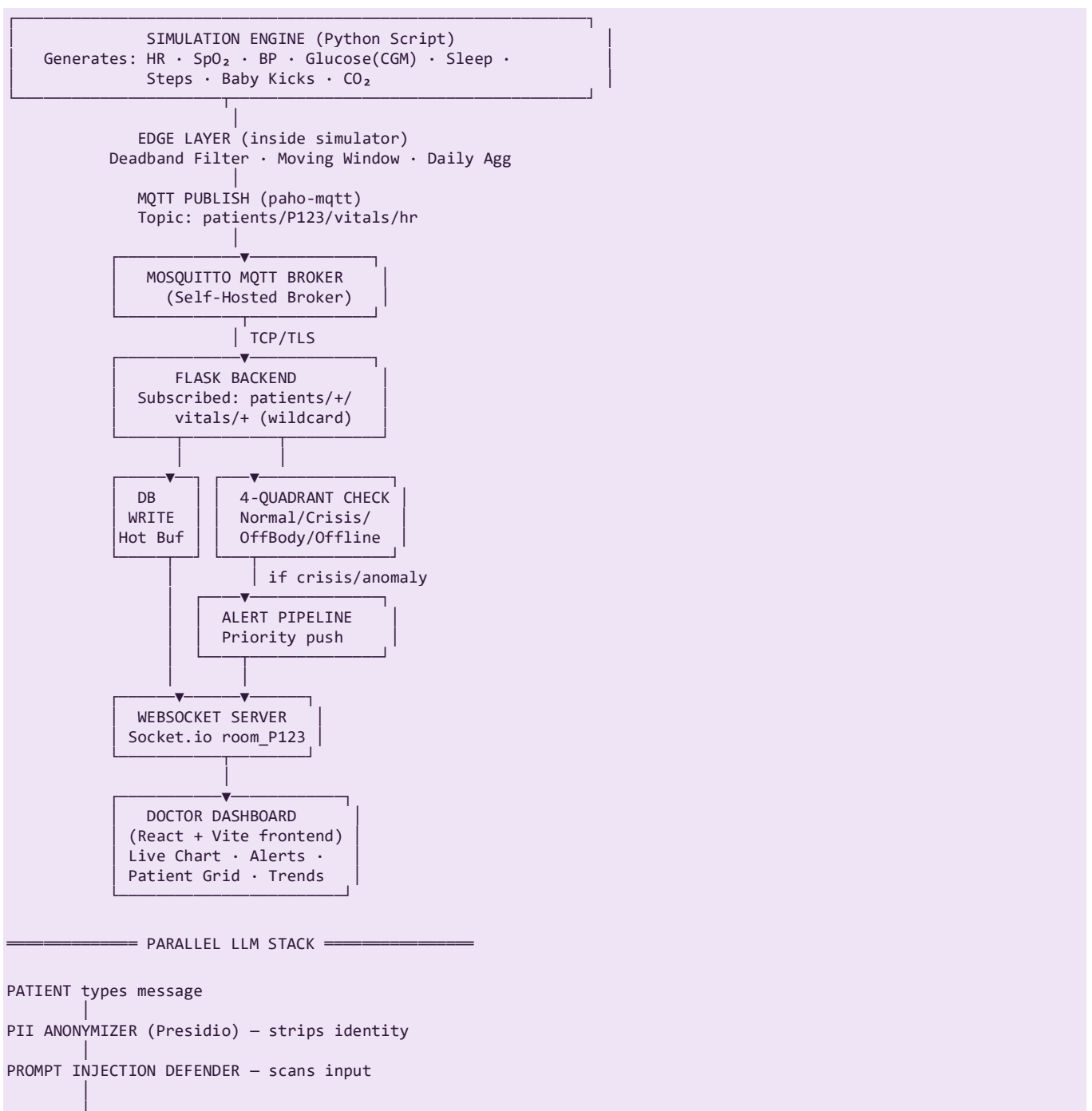
*Maternal Health Platform*

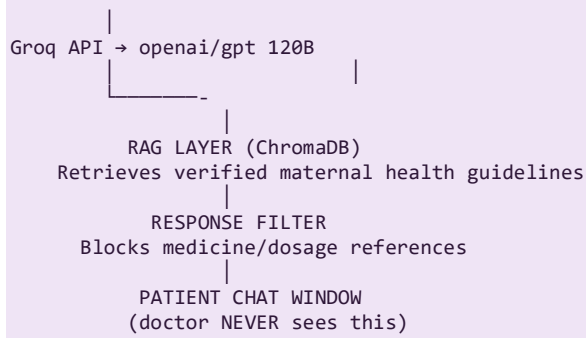
Updated Tech Stack — React + Vite · Flask · MySQL · Mosquitto MQTT

## NOTE: THE BACKGROUND WORKER- REDIS RQ

### The Master Flow at a Glance

Every piece of data in this system travels through 8 distinct layers before it reaches the doctor's screen. The LLM stack runs as a completely parallel, isolated pipeline. The structure and logic below are unchanged from the original design — only the underlying technology at each layer has been swapped.





## Layer 1 — The Simulation Engine

*Where patient vitals are generated — the origin of all data*

The simulation engine is a Python script running as a background worker on a cloud server (Render or Railway). It acts as the software equivalent of the patient's wearable IoT device. It generates realistic physiological values for every registered patient whose wearable is in the "Connected" state. This layer is unchanged — it has no dependency on the frontend, backend, database, or MQTT broker choice.

### What the simulator generates per patient

| Parameter                 | Normal range               | Generation method                | Raw update frequency   |
|---------------------------|----------------------------|----------------------------------|------------------------|
| Heart Rate                | 70–115 bpm                 | random.randint(70,115)           | Every 30 seconds       |
| SpO <sub>2</sub> (Oxygen) | 95–99%                     | random.randint(95,99)            | Every 30 seconds       |
| Blood Glucose (CGM)       | 70–95 mg/dL fasting        | random.uniform(70,95)            | Every 60 seconds       |
| Blood Pressure            | 110–130 / 70–85 mmHg       | moving avg of 3 readings locally | Every 3 minutes        |
| Baby Kicks                | 0–3 per session            | random event probability         | Once or twice daily    |
| Sleep Quality             | Score 0–100                | algorithm on HR + motion         | Once daily at midnight |
| Physical Activity/Steps   | 0–15000 steps              | accumulator integer              | Once daily at midnight |
| CO <sub>2</sub> Level     | Normal environmental range | random.uniform(400,600) ppm      | Every 30 seconds       |

### The 4-Quadrant State Check (runs before every packet is generated)

Before generating any value, the simulator checks what state the device is in. Unchanged from the original design:

```
def generate_packet(patient_id):
    state = check_device_state(patient_id)

    # Rule 1: Table state is supreme
    if state in ["table_manual", "table_auto"]:
        return {"vitals": None, "status": "sensor_off_body"}

    # Rule 2: Crisis override (manual beats automatic)
    if state == "crisis_manual":
        return generate_emergency_values(patient_id)

    # Rule 3: Autonomous stochastic spike (5% dice roll)
    if random.randint(1,100) <= 5:
        return generate_emergency_values(patient_id)

    # Default: normal healthy values
```

```
return generate_normal_values(patient_id)
```

## Layer 2 — The Edge Layer

---

*Smart local processing inside the simulator — before any network call*

The edge layer runs inside the simulator script itself, before any data is published to MQTT. It decides what is worth sending, what can be discarded, and what must be sent immediately. This layer is pure Python logic and is completely unaffected by the tech stack swap.

### Step 1: Deadband Filter — Heart Rate, SpO<sub>2</sub>, Blood Glucose

```
last_hr = 72
last_spo2 = 97
last_glucose = 88

def should_transmit_continuous(metric, new_val, last_val, threshold, danger_low, danger_high):
    # Always transmit if danger zone crossed
    if new_val <= danger_low or new_val >= danger_high:
        return True, "CRITICAL"

    # Transmit if changed beyond deadband threshold
    if abs(new_val - last_val) >= threshold:
        return True, "CHANGE"

    # Check 5-minute heartbeat rule
    if time_since_last_tx > 300:
        return True, "HEARTBEAT"

    # Suppress – duplicate of last value
    return False, "SUPPRESSED"

# Thresholds used:
# Heart Rate: deadband=10 bpm, danger_low=50, danger_high=120
# SpO2: deadband=2%, danger_low=94, danger_high=None
# Blood Glucose: deadband=5 mg/dL, danger_low=70, danger_high=140
```

**What gets suppressed:** HR reading of 72 when last was 72 — suppressed. HR reading of 85 when last was 72 — transmitted (change of 13 exceeds deadband of 10). Glucose at 88 when last was 90 — suppressed (change of 2 is under deadband of 5).

### Step 2: Moving Window Average — Blood Pressure

```
bp_local_buffer = []

def edge_process_bp(systolic, diastolic):
    # Emergency bypass – transmit immediately
    if systolic > 160:
        mqtt_publish_immediately({
            "bp_systolic": systolic,
            "bp_diastolic": diastolic,
            "priority": "CRITICAL"
        })
        bp_local_buffer.clear()
        return

    # Add to local buffer
    bp_local_buffer.append((systolic, diastolic))

    # After 3 readings over ~3 minutes, send smoothed average
    if len(bp_local_buffer) >= 3:
```

```

    avg_sys = sum(r[0] for r in bp_local_buffer) / 3
    avg_dia = sum(r[1] for r in bp_local_buffer) / 3
    mqtt_publish({
        "bp_systolic": round(avg_sys, 1),
        "bp_diastolic": round(avg_dia, 1),
        "type": "smoothed_average"
    })
    bp_local_buffer.clear()

```

### Step 3: Daily Aggregation — Sleep, Steps, Baby Kicks

```

daily_steps = 0
sleep_segments = []
kick_count = 0

def record_step_event(steps_this_minute):
    daily_steps += steps_this_minute

def record_sleep_segment(quality_score, duration_mins):
    sleep_segments.append((quality_score, duration_mins))

def record_kick():
    kick_count += 1

# At midnight – compute and publish one daily summary packet
def publish_daily_summary(patient_id):
    total_sleep_score = calculate_sleep_score(sleep_segments)
    mqtt_publish({
        "patient_id": patient_id,
        "type": "daily_summary",
        "total_steps": daily_steps,
        "sleep_score": total_sleep_score,
        "kick_count": kick_count,
        "timestamp": midnight_timestamp
    })
    # Reset for next day
    daily_steps = 0
    sleep_segments = []
    kick_count = 0

```

## Layer 3 — MQTT Publish — The Transport Layer

*How the simulator sends data to the broker*

Once the edge layer decides a packet is worth sending, it is published to the MQTT broker using the paho-mqtt Python library — the same library works identically whether the broker behind it is HiveMQ Cloud or a self-hosted Mosquitto instance, since both speak standard MQTT. Only the connection target and credentials change.

### Topic structure — how data is organised (unchanged)

```

# Every packet goes to a specific topic path
# Format: patients/{patient_id}/vitals/{parameter}

patients/P123/vitals/hr      -> {"value": 82, "ts": 1719123456}
patients/P123/vitals/spo2    -> {"value": 97, "ts": 1719123456}
patients/P123/vitals/glucose -> {"value": 88, "ts": 1719123456}
patients/P123/vitals/bp      -> {"systolic": 118, "diastolic": 76, "ts": ...}
patients/P123/vitals/daily   -> {"steps": 8420, "sleep": 82, "kicks": 12}
patients/P123/status/device  -> {"state": "connected", "battery": 87}
patients/P123/alert/critical -> {"type": "bp_spike", "value": 162, "ts": ...}

```

## The publish code — updated for Mosquitto

```
import paho.mqtt.client as mqtt
import json

client = mqtt.Client()
client.tls_set() # TLS encryption (recommended even for self-hosted)
client.username_pw_set("username", "password") # Mosquitto auth file
client.connect("your-mosquitto-broker-ip", 8883) # Self-hosted Mosquitto

def mqtt_publish(topic, payload, priority="normal"):
    qos = 2 if priority == "CRITICAL" else 1
    # QoS 2 = exactly once delivery (critical alerts)
    # QoS 1 = at least once delivery (normal readings)
    client.publish(
        topic=topic,
        payload=json.dumps(payload),
        qos=qos,
        retain=False
    )
```

**QoS levels explained:** QoS 0 = fire and forget (fastest, may lose packets). QoS 1 = at least once (normal vitals). QoS 2 = exactly once, guaranteed delivery (critical alerts — BP spike, SpO<sub>2</sub> crash). Critical alerts always use QoS 2 — no missed deliveries on emergencies. This behaviour is identical on Mosquitto, since QoS is part of the MQTT protocol itself, not broker-specific.

## Layer 4 — Mosquitto MQTT Broker

---

*The message routing hub between simulator and backend*

Mosquitto is the MQTT broker — the central routing hub. It receives every packet from the simulator and forwards it to any subscriber that has requested that topic pattern. The Flask backend maintains a permanent connection to this broker, exactly as it did with HiveMQ Cloud — the only difference is that this broker now runs on infrastructure you control (e.g. a small always-on cloud VM) rather than a third-party managed service.

### What the broker does

- Receives packets from the simulator on topic patients/P123/vitals/hr.
- Checks its subscriber list — which connected clients are listening to this topic?
- Immediately forwards the packet to all matching subscribers.
- Handles thousands of concurrent connections without data crossing between patients.
- Mosquitto has no built-in connection cap — unlike HiveMQ Cloud's free-tier 100-connection limit, the real ceiling is the hardware of the VM it runs on, which comfortably handles far more than a prototype's 3 to 5 test patients.

### The wildcard subscription pattern (unchanged)

```
# The backend subscribes to ALL patient topics at once
# using a single wildcard pattern

backend.subscribe("patients/+/vitals/+")

# The + wildcard matches any single level
# patients/P123/vitals/hr    matches
# patients/P124/vitals/bp    matches
# patients/P125/vitals/daily matches

# The broker routes each packet to the backend
# The backend then sorts by patient_id internally
```

## Layer 5 — Flask Backend — The Brain

---

*Parsing, validation, database writing, and WebSocket routing*

The Flask backend is the central intelligence of the system, replacing FastAPI. This is the layer with the most real code changes: Flask is synchronous by default (no `async/await`), and the database driver changes from Postgres's `asyncpg` to a MySQL connector. The logic — parse packet, check thresholds, write to database, push to WebSocket — is identical; only how each step is written changes.

### Step 1: Receive and Parse the MQTT Packet

```
import json
from flask import Flask
import paho.mqtt.client as mqtt

app = Flask(__name__)

# Backend subscribes to broker on startup
def on_message(client, userdata, message):
    topic = message.topic
    payload = json.loads(message.payload.decode())

    # Extract patient_id from topic
    # "patients/P123/vitals/hr" -> patient_id = "P123"
    parts = topic.split("/")
    patient_id = parts[1]
    param_type = parts[3]

    # Route to handler — no asyncio needed, paho-mqtt's
    # network loop already runs this callback on its own thread
    handle_vital(patient_id, param_type, payload)
```

### Step 2: The 4-Quadrant State Check and Threshold Validation

```
def handle_vital(patient_id, param_type, payload):

    # Check for off-body state first (supreme rule)
    if payload.get("status") == "sensor_off_body":
        handle_off_body(patient_id)
        return

    # Check anomaly detection (Isolation Forest ML model)
    is_anomaly, reason = anomaly_detector.check(patient_id, payload)

    # Check hard thresholds
    is_critical = check_thresholds(payload)

    # Determine routing priority
    if is_critical or is_anomaly:
        priority = "CRITICAL"
    else:
        priority = "NORMAL"

    # Proceed to database write
    write_to_database(patient_id, param_type, payload, priority)
    push_to_websocket(patient_id, payload, priority)

def check_thresholds(payload):
    if payload.get("hr", 0) > 120: return True
    if payload.get("spo2", 100) < 94: return True
```

```

if payload.get("bp_systolic", 0) > 140: return True
if payload.get("glucose", 0) > 140: return True
if payload.get("glucose", 999) < 70: return True
return False

```

### Step 3: Database Write — Three-Tier Storage (now on MySQL)

Postgres's asyncpg driver and its \$1, \$2 positional placeholders are replaced with a MySQL connector using %s placeholders. The three-tier Hot Buffer / Warm Trend / Cold Ledger structure and cron-based compression logic are unchanged.

```

import mysql.connector

db = mysql.connector.connect(
    host="your-mysql-host", user="...", password="...", database="maternal_health"
)

def write_to_database(patient_id, param_type, payload, priority):
    cursor = db.cursor()

    # -- TIER 1: Hot Buffer (live chart data – deleted after 24h) --
    cursor.execute("""
        INSERT INTO vitals_hot_buffer
        (patient_id, param_type, value, recorded_at, priority)
        VALUES (%s, %s, %s, NOW(), %s)
        """, (patient_id, param_type, payload["value"], priority))
    db.commit()
    cursor.close()

    # -- Midnight cron compresses hot -> warm --
    # (runs automatically via a scheduled cron job)

    # -- Midnight cron compresses warm -> cold --
    # (runs weekly, produces one row per day)

    # -- TIER 2: Warm Trend (hourly baselines – 7-day view) --
    # Written by cron job at end of each hour
    # SELECT AVG(value) FROM vitals_hot_buffer
    # WHERE patient_id=%s AND recorded_at > NOW() - INTERVAL 1 HOUR

    # -- TIER 3: Cold Ledger (daily average – 9-month history) --
    # Written by cron job at midnight
    # SELECT AVG(value) FROM vitals_warm_trend
    # WHERE patient_id=%s AND date = CURDATE()

```

**Syntax note:** MySQL's NOW() and date functions like CURDATE() replace Postgres's NOW() and interval syntax (NOW() - 1 hour becomes NOW() - INTERVAL 1 HOUR in MySQL). Parameter placeholders change from Postgres's numbered \$1/\$2 style to MySQL's positional %s style — the query logic itself is unchanged.

## Layer 6 — WebSocket Server — The Real-Time Pipe

*Flask-SocketIO rooms maintaining persistent connections to each doctor*

The WebSocket server maintains a permanent, open connection to every doctor's browser that is currently logged in — unchanged in behaviour. The library changes from python-socketio's AsyncServer (built for FastAPI's ASGI model) to Flask-SocketIO, which integrates with Flask's synchronous request model. The room-per-patient isolation logic is identical.

### How doctor authentication and room assignment works

```

from flask_socketio import SocketIO, join_room, emit

```

```

socketio = SocketIO(app, cors_allowed_origins="*")
socket_doctor_map = {}

# When doctor opens a patient's dashboard

@socketio.on("connect")
def handle_connect(auth):
    # Verify JWT token
    token = auth.get("token") if auth else None
    doctor_id = verify_jwt(token)

    if not doctor_id:
        return False # rejects the connection

    # Store which doctor this socket belongs to
    socket_doctor_map[request.sid] = doctor_id

@socketio.on("watch_patient")
def handle_watch_patient(data):
    patient_id = data["patient_id"]
    doctor_id = socket_doctor_map[request.sid]

    # Zero Trust check – is this doctor authorized for this patient?
    cursor = db.cursor()
    cursor.execute("""
        SELECT 1 FROM patient_doctor_assignments
        WHERE patient_id=%s AND doctor_id=%s
    """, (patient_id, doctor_id))
    authorized = cursor.fetchone()
    cursor.close()

    if not authorized:
        emit("error", {"code": 403, "message": "Access denied"})
        return

    # Join the private room for this patient
    join_room(f"room_{patient_id}")
    emit("subscribed", {
        "patient_id": patient_id,
        "message": "Now receiving live vitals"
    })

```

### Multi-patient isolation — how N doctors see N patients simultaneously (unchanged)

```

# Doctor A watches Patient P123
Doctor_A_socket -> joins room_P123

# Doctor B watches Patient P124
Doctor_B_socket -> joins room_P124

# When P123 data arrives:
socketio.emit("vitals_update", data, room="room_P123")
-> Only Doctor A receives this
-> Doctor B receives nothing

# When P124 data arrives:
socketio.emit("vitals_update", data, room="room_P124")
-> Only Doctor B receives this
-> Doctor A receives nothing

```



```
# Zero data leakage between patients
```

The `push_to_websocket` step, called from Layer 5 after every database write, becomes:

```
def push_to_websocket(patient_id, payload, priority):
    cursor = db.cursor()
    cursor.execute("""
        SELECT doctor_id FROM patient_doctor_assignments
        WHERE patient_id = %s
    """, (patient_id,))
    row = cursor.fetchone()
    cursor.close()

    room_name = f"room_{patient_id}"

    if priority == "CRITICAL":
        socketio.emit("critical_alert", {
            "patient_id": patient_id,
            "data": payload,
            "reason": determine_alert_reason(payload)
        }, room=room_name)
    else:
        socketio.emit("vitals_update", {
            "patient_id": patient_id,
            "data": payload
        }, room=room_name)

    # Only the doctor authorized for this patient
    # is in room_{patient_id}
    # Doctor B cannot receive Patient A's data
```

## Layer 7 — Doctor Dashboard — The Frontend

---

*How the browser renders live data — now built with React + Vite*

The doctor's browser maintains an active WebSocket connection to the server. This layer is almost entirely unchanged — `socket.io-client` works identically regardless of whether the app was bootstrapped with Create React App or Vite, since it's a browser-side JavaScript library talking over the same Socket.io protocol that Flask-SocketIO speaks on the server. The one real difference is how environment variables like the backend URL are read.

### Step 1: Receiving WebSocket Events in the Browser

```
// Doctor's browser — JavaScript (React + Vite)

import { io } from "socket.io-client"

// Vite exposes env vars via import.meta.env, prefixed with VITE_
const socket = io(import.meta.env.VITE_BACKEND_URL, {
  auth: { token: localStorage.getItem("jwt_token") }
})

// Subscribe to a patient's live feed
socket.emit("watch_patient", { patient_id: "P123" })

// Handle normal vitals update
socket.on("vitals_update", (data) => {
  updateChartLine(data.param_type, data.value, data.timestamp)
  updateStatusCard(data.patient_id, data.value)
})

// Handle critical alert — highest priority
socket.on("critical_alert", (data) => {
```

```

    flashRedBanner(data.patient_id, data.reason)
    playAlertSound()
    showOverlayModal(data)
    updatePatientGridToRed(data.patient_id)
  })

  // Handle off-body sensor state
  socket.on("sensor_off_body", (data) => {
    flatlineChart(data.patient_id)
    showYellowBanner("Sensor Off-Body / No Signal Detected")
    keepConnectivityLightGreen() // device is still powered on
  })

```

**Vite environment variables:** Create React App reads `process.env.REACT_APP_*`; Vite reads `import.meta.env.VITE_*` instead, and variable names must be prefixed with `VITE_` to be exposed to browser code. A `.env` file in the project root with `VITE_BACKEND_URL=https://your-backend-host` would supply this value.

## Step 2: The Three Dashboard UI Patterns (unchanged)

### Pattern 1 — Exception-Based Alerts Inbox

- Full-screen red overlay flashes instantly.
- Audio chime fires — even if browser is minimised.
- Specific offending metric and value displayed prominently.
- Alert logged to the Alerts Inbox with timestamp.

### Pattern 2 — Triage Patient Grid (Color-Coded)

- Green card — Patient Sarah: All vitals normal. Metrics hidden in dropdown. Screen stays clean.
- Red card — Patient Priya: High risk. Row expands automatically showing only the specific offending metric — e.g., "Blood Pressure: 145/95".
- Yellow card — Sensor off-body. Device powered on but not worn.
- Grey card — Device offline / disconnected.

Pattern 3 — On-Demand Trend Analytics. When the doctor clicks "View Analytics Chart" for a patient, the browser sends an API GET request to pull historical data from the correct database tier:

```

// Fetch last 24 hours — from Hot Buffer
GET /api/patient/P123/vitals?range=24h

// Fetch last 7 days — from Warm Trend
GET /api/patient/P123/vitals?range=7d

// Fetch full 9-month pregnancy — from Cold Ledger
GET /api/patient/P123/vitals?range=9m

```

## Step 3: Partial History — The Day 3 Chart Rendering (unchanged)

```

// Patient enrolled 3 days ago — 7-day chart requested

const chartData = await fetchVitalsHistory("P123", "7d")
// Returns only 3 days of real data

// Chart.js handles partial data gracefully
const labels = ["Day -4", "Day -3", "Day -2", "Day -1", "Day 1", "Day 2", "Day 3"]
const values = [null, null, null, null, 82, 79, 81]
// <- 4 days N/A -> <- 3 real days ->

// Render onboarding anchor line
annotations: {
  onboardingLine: {
    type: "line",
    x: "Day 1",

```

```

        label: "Device Inception / Onboarding Date"
    }
}

// Calculate average using only enrolled days
const average = sum(values.filter(v => v !== null)) / enrolledDays
// NOT divided by 7 – divided by 3

```

## Layer 8 — The LLM Stack — Parallel Pipeline

---

*How the AI lifestyle assistant processes patient messages — completely isolated from doctor*

The LLM pipeline runs completely in parallel with the vitals pipeline and has no connection to the doctor's dashboard. This entire stack is Python-based and framework-agnostic — Presidio, the injection scanner, ChromaDB, Ollama, and Groq all work identically under Flask as they did under FastAPI. The only line that changes is the database query in Step 8, updated to MySQL syntax.

### Step 1: Patient Types a Message — PII Anonymizer Runs First

```

from presidio_analyzer import AnalyzerEngine
from presidio_anonymizer import AnonymizerEngine

analyzer = AnalyzerEngine()
anonymizer = AnonymizerEngine()

def anonymize(text):
    results = analyzer.analyze(text=text, language="en")
    return anonymizer.anonymize(text=text, analyzer_results=results).text

# Example:
# Input: "My name is Priya and I live in Anna Nagar, BP was 138 today"
# Output: "My name is [NAME] and I live in [LOCATION], BP was 138 today"
#
# Identity stripped. Clinical context preserved.
# This anonymised version is what all downstream systems see.

```

### Step 2: Prompt Injection Defender Scans the Input

```

INJECTION_PATTERNS = [
    "ignore all previous instructions",
    "forget your system prompt",
    "you are now",
    "jailbreak",
    "DAN mode",
]

BOUNDARY_VIOLATIONS = [
    "dosage", "mg", "tablet", "prescri",
    "stop taking", "overdose"
]

def scan(text):
    for pattern in INJECTION_PATTERNS:
        if pattern in text.lower():
            return False, "prompt_injection"
    for pattern in BOUNDARY_VIOLATIONS:
        if pattern in text.lower():
            return False, "boundary_violation"
    return True, None

# If blocked -> return safe response without touching LLM

```

```
# If safe -> proceed to router
```

#### Step 4: RAG Layer — Retrieve Verified Medical Context

```
from langchain.vectorstores import Chroma
from langchain.embeddings import HuggingFaceEmbeddings

embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)
vectordb = Chroma(
    persist_directory="./chroma_db",
    embedding_function=embeddings
)

def retrieve_context(question, k=3):
    results = vectordb.similarity_search(question, k=k)
    return "\n\n".join([doc.page_content for doc in results])

else:
    # Groq API – openai/gpt 120B – anonymised input only
    response = groq_client.chat.completions.create(
        model="openai/gpt 120b-versatile",
        messages=messages
    )
    return response.choices[0].message.content, "groq_cloud"
```

#### Step 6: System Prompt — What the LLM Is Told Before Every Response

```
def build_system_prompt(patient_profile, rag_context):
    return f"""
You are a maternal health lifestyle assistant.

PATIENT PROFILE:
Trimester: {patient_profile["trimester"]}
Conditions: {patient_profile["conditions"]}
Job type: {patient_profile["job_type"]}

VERIFIED MEDICAL GUIDELINES (from your knowledge base):
{rag_context}

STRICT RULES:
- Never suggest any medicine, tablet, or dosage
- Never diagnose any symptom or condition
- Never answer questions outside maternal health
- Always end food/exercise advice with the safety disclaimer
- If patient reports serious symptom: tell her to contact doctor directly
- You have NO connection to the doctor – you cannot notify anyone
"""
```

#### Step 7: Response Filter — Final Safety Check Before Delivery

```
BLOCKED_OUTPUT_PHRASES = [
    "mg", "milligram", "dosage", "take this",
    "stop eating", "stop taking", "you have",
    "diagnosed with", "you should stop"
]

def filter_response(llm_output):
    for phrase in BLOCKED_OUTPUT_PHRASES:
        if phrase in llm_output.lower():
            return ("I want to be careful here. This is something your doctor "
                    "should advise on directly. Please use the consultation section to reach them.")
    return llm_output # safe – deliver to patient
```

```
# The filtered response goes ONLY to the patient chat window
# It is NEVER forwarded to the doctor dashboard
# It is saved to an encrypted patient-only chat log table
```

## Step 8: On-Demand Vitals Snapshot — The Blind Vitals Rule

By default the LLM is completely blind to the patient's vitals database. If the patient clicks "Attach My Current Vitals", the following happens — now querying MySQL:

```
# Patient clicks "Attach My Current Vitals"

def attach_vitals_snapshot(patient_id):
    cursor = db.cursor(dictionary=True)
    cursor.execute("""
        SELECT hr, spo2, bp_systolic, bp_diastolic, glucose, recorded_at
        FROM vitals_hot_buffer
        WHERE patient_id = %s
        ORDER BY recorded_at DESC LIMIT 1
    """, (patient_id,))
    latest = cursor.fetchone()
    cursor.close()

    # Convert to plain text snapshot
    snapshot = f"""
Current vitals as of {latest["recorded_at"]}:
Heart Rate: {latest["hr"]} bpm
SpO2: {latest["spo2"]} %
BP: {latest["bp_systolic"]}/{latest["bp_diastolic"]} mmHg
Glucose: {latest["glucose"]} mg/dL
    """

    # Pass snapshot to LLM ONCE for this message only
    # After LLM responds – snapshot is wiped from context
    # Doctor dashboard is NEVER informed of this interaction
    return snapshot
```

## Complete End-to-End Flow — Step by Step in Plain Words

Here is the entire journey of a single heart rate reading — from the moment the simulator generates it to the moment it appears on the doctor's screen. The steps and decision logic are unchanged from the original design; only the named technologies at each step reflect the new stack.

### A normal, non-critical reading

| # | Layer                    | What happens                                                                                                                           |
|---|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Simulator generates      | Python script creates HR value of 82 bpm for Patient P123. Checks 4-quadrant state — device is worn, no crisis. Proceeds normally.     |
| 2 | Edge — Deadband check    | Last transmitted HR was 79 bpm. Change is 3 bpm — below deadband of 10. Packet suppressed. Not sent. Chart holds 79 on doctor screen.  |
| 3 | Edge — Heartbeat check   | Time since last transmission is 4 minutes 40 seconds. Under 5-minute threshold. Suppression holds.                                     |
| 4 | 5-minute heartbeat fires | Time exceeds 300 seconds. Force transmit fires. Packet published to MQTT with HR 82 and type = "heartbeat".                            |
| 5 | MQTT publish             | Packet published to topic patients/P123/vitals/hr on the Mosquitto broker. QoS 1 (at least once).                                      |
| 6 | Mosquitto routes         | Broker receives packet. Checks subscriber list. Backend is subscribed to patients/+ /vitals/+. Forwards packet to backend via TCP/TLS. |

| #  | Layer                    | What happens                                                                                                                                                                                      |
|----|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7  | Flask receives           | Backend on_message fires. Parses patient_id = P123, param_type = hr, value = 82. Routes to handle_vital().                                                                                        |
| 8  | Threshold check          | 82 bpm is below danger threshold of 120. is_critical = False. Anomaly detector checks against P123 personal baseline. No anomaly. priority = NORMAL.                                              |
| 9  | Hot Buffer write (MySQL) | Row inserted into vitals_hot_buffer: patient_id=P123, param=hr, value=82, recorded_at=NOW(). A scheduled cleanup job will delete this row after 24 hours.                                         |
| 10 | WebSocket push           | Backend emits vitals_update event via Flask-SocketIO to room room_P123 with payload {hr: 82, ts: ...}.                                                                                            |
| 11 | Doctor browser receives  | Doctor A's browser receives the socket event. Chart.js updates the heart rate line to 82. Status card shows green. No alert. Dashboard is quiet.                                                  |
| 12 | Doctor opens trend chart | Later, Doctor A clicks "View 7-day Analytics". Browser sends GET /api/patient/P123/vitals?range=7d. Backend queries Warm Trend table. Returns hourly averages. Chart renders 7-day HR trend line. |

### What happens when HR spikes to 135 bpm (crisis)

| # | Layer                     | What happens                                                                                                                                                                                       |
|---|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Simulator generates       | HR 135 bpm. Deadband check: change of 53 bpm exceeds threshold of 10. Danger check: 135 > 120 threshold. Immediate transmit — QoS 2.                                                               |
| 2 | MQTT publish              | Packet published with priority = CRITICAL to topic patients/P123/alert/critical on Mosquitto. QoS 2 guarantees exactly-once delivery.                                                              |
| 3 | Flask receives            | is_critical = True. Skips normal flow. Routes directly to alert pipeline.                                                                                                                          |
| 4 | Database write (MySQL)    | Row inserted into Hot Buffer with priority = CRITICAL. Also inserts into alerts_log table with timestamp.                                                                                          |
| 5 | WebSocket push — priority | Backend emits critical_alert event via Flask-SocketIO to room_P123. Does not wait for next cycle.                                                                                                  |
| 6 | Doctor browser            | socket.on("critical_alert") fires. Red banner flashes. Audio alert plays. Overlay modal shows: "Patient P123 — HR 135 bpm — Above safe threshold of 120". Patient grid card turns red immediately. |

### What happens when patient asks the LLM a question

| # | Layer                  | What happens                                                                                                                               |
|---|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Patient types          | Patient types: "My name is Priya, I have diabetes, I am 28 weeks pregnant, what should I eat for breakfast?"                               |
| 2 | PII Anonymizer         | Presidio detects [NAME]. Output: "My name is [NAME], I have diabetes, I am 28 weeks pregnant, what should I eat for breakfast?"            |
| 3 | Prompt injection scan  | No injection patterns detected. Input is safe. Proceed.                                                                                    |
|   |                        |                                                                                                                                            |
| 4 | RAG retrieves context  | ChromaDB searches for "gestational diabetes breakfast foods". Returns 3 relevant chunks from ICMR dietary guidelines.                      |
| 5 | System prompt built    | Patient profile injected: Trimester 2, gestational diabetes, desk job. RAG context appended. Full system prompt constructed.               |
| 6 |                        |                                                                                                                                            |
| 7 | Response filter        | No blocked phrases detected. Response is safe. Delivered to patient chat window.                                                           |
| 8 | Chat log saved (MySQL) | Anonymised message and response saved to patient_chat_logs table (encrypted). Doctor dashboard receives nothing. Doctor is never notified. |

## Summary of the tech stack swap

| Layer                                                                                      | Original                                   | Updated                                  |
|--------------------------------------------------------------------------------------------|--------------------------------------------|------------------------------------------|
| Frontend                                                                                   | React                                      | React + Vite                             |
| Backend                                                                                    | FastAPI (async)                            | Flask (sync) + Flask-SocketIO            |
| Database                                                                                   | PostgreSQL (asyncpg, \$1 params)           | MySQL (mysql-connector, %s params)       |
| MQTT Broker                                                                                | HiveMQ Cloud (managed, 100-connection cap) | Mosquitto (self-hosted, no built-in cap) |
| Everything else — simulation engine, edge logic, LLM stack (Presidio, RAG/ChromaDB, /Groq) | Unchanged                                  | Unchanged                                |